Approach Document — SHL Assessment Recommendation Agent

Architecture

The agent runs one explicit, linear pipeline per stateless `/chat` call — no LangChain Agent, no LangGraph, no autonomous loop:

Conversation Analyzer → Intent Detection → Missing-Info Detector → Decision Engine → {Retriever + Recommendation Generator | Compare | Clarify | Refuse} → Validator → JSON Formatter

FastAPI routes are thin (DI + orchestrator call only). Business logic lives in `app/agent`, `app/services`, `app/retriever`, and `app/ranking`, each with one responsibility, per the assignment's modularity requirement. Since the API is stateless, `ConversationState` is rebuilt from the full message history on every call — no session cache, no DB row.

Retrieval strategy

Hybrid retrieval fuses three signals over the 377-item Individual Test Solutions catalog (scraped and supplied for this assignment, reconstructed into structured JSON in `scripts/build_catalog.py`):

- **Semantic search** (ChromaDB, `BAAI/bge-small-en-v1.5` via Sentence

Transformers, local/free) — catches paraphrased intent ("someone who can lead under pressure" → personality assessments).

- **Keyword search** (BM25) — catches exact product codes and acronyms

("OPQ32r", ".NET MVC") that a general-purpose embedding model wasn't trained to distinguish precisely.

- **Metadata fit** — scores job level, test type, language, duration, and

adaptive/remote against constraints the user actually stated (never a penalty for constraints they didn't mention).

All three scores are normalized to `[0, 1]` and combined by weighted sum (`app/ranking/rank_fusion.py`), not Reciprocal Rank Fusion — the scores are already comparable, and a weighted sum is more interpretable and tunable against the 10 labeled traces than RRF's rank-only blending.

Prompt strategy

One `PromptTemplate` per behavior — Clarification, Recommendation, Refinement, Comparison, Refusal, and Constraint Extraction (`app/prompts/templates.py`) — sharing a single grounding-rules block rather than duplicating hallucination-prevention wording six times or collapsing into one mega-prompt. Every generation prompt is handed only the retrieved catalog snippets and is instructed to reference names/URLs verbatim from that context. Clarification is explicitly instructed to ask **one** high-value question rather than a list, per the assignment's guidance.

Hallucination prevention is enforced twice, and only one of them is trusted: the prompt asks the LLM not to invent names or URLs, but the `Validator` (`app/agent/validator.py`) is the actual guarantee — every recommended URL is checked against `CatalogRepository.is_valid_url` and silently dropped if it doesn't match a real catalog entry. This is the layer the hard-eval scoring ("items from catalog only") is graded against, so it cannot be advisory-only.

Decision logic

Intent detection (`app/agent/intent_detector.py`) is regex-based and deterministic — refusal (off-topic / prompt-injection patterns), comparison ("compare", "difference between", or two named catalog assessments), refinement ("actually", "instead", "also add" — only after a prior assistant turn), else recommend-or-clarify. The Decision Engine (`app/agent/decision_engine.py`) is pure Python with zero LLM calls: it picks one action and computes `end_of_conversation`, respecting the 8-turn cap (forcing a recommendation over another clarifying question once only one turn remains). This keeps the highest-stakes, most failure-prone logic unit-testable without mocking an LLM at all (`tests/test_decision_engine.py`, `tests/test_intent_detector.py`).

Evaluation

`scripts/run_eval.py` replays the 10 supplied labeled traces against a running instance, computing Recall@10 per the assignment's own definition (the "expected shortlist" is parsed from each trace's final `end_of_conversation: true` turn), plus schema-compliance and turn-cap checks on every response. `app/evaluation/probes.py` runs four behavior probes: no recommendation on a vague turn 1, off-topic refusal, prompt-injection resistance, and refinement honored without restarting the shortlist. Unit tests additionally cover the catalog loader, hybrid retriever, decision engine, intent detector, and prompt rendering in isolation.

What didn't work / was revised: an earlier version scored comparison intent purely on the word "compare", which missed direct-name comparisons like "OPQ32r vs GSA"; `mentions_two_or_more_known_assessments` was added to catch those. An earlier metadata scorer penalized assessments for *unstated* constraints (e.g. treating "no adaptive flag mentioned" as a mismatch), which suppressed otherwise-good matches — it was changed to contribute `0` (neutral) for anything the user never stated, only scoring constraints actually present in the conversation.

Tradeoffs

- **Weighted-sum fusion over RRF**: more interpretable and tunable, less

theoretically robust to wildly different score distributions across future retrieval signals.

- **Regex-based intent/refusal detection over an LLM classifier**: faster,

deterministic, zero extra latency/cost, and testable without mocking — at the cost of not generalizing to phrasings outside the pattern set (a known limitation, mitigated by keeping patterns broad and reviewing behavior probes rather than chasing perfect recall on refusal phrasing).

- **Single LLM call for constraint extraction per turn**, with a

regex-free JSON-parse fallback to an empty profile on failure — prioritizes availability (never 500s on a parsing hiccup) over always having perfect constraint recall on that turn.

- **Replay harness uses verbatim recorded user turns**, not a second

LLM-simulated persona — a fully deterministic, dependency-free approximation for local iteration; SHL's own evaluator does the fuller LLM-simulated-user replay.

AI tools used

Claude (Anthropic) was used as an AI pair-programmer for the full implementation — architecture scaffolding, module code, prompt drafting, and this document — under an explicit engineering brief the developer authored and reviewed line-by-line. The developer owns the design decisions above and can walk through any module in the technical deep-dive.

Limitations & future improvements

No cross-encoder re-ranking of top candidates (latency/Recall@10 tradeoff to evaluate against the 30s budget); comparison-intent name resolution is substring-based and won't resolve category-only comparisons ("a personality test vs. an ability test"); no response caching for repeated identical queries; only one LLM provider adapter is implemented today, though the `LLMClient` protocol supports adding more without touching agent/service code.